МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Кафедра комп'ютерної інженерії

Дисципліна: Програмування
Спеціальність: 123 «Комп’ютерна інженерія»
Освітній рівень: Перший (бакалаврський)
Семестр: I
Варіант: 3

ЛАБОРАТОРНА РОБОТА № 4

Тема: Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

1. МЕТА РОБОТИ

- Опанувати принципи функціональної декомпозиції програм: проектування чистих підпрограм із чітким розділенням на прототипи (оголошення) та реалізацію (визначення).
-Дослідити механізм роботи автоматичних локальних змінних на стеку та використання специфікатора static для збереження внутрішнього стану функцій між послідовними викликами.
-Опанувати низькорівневе маніпулювання даними на рівні окремих розрядів за допомогою порозрядних операцій & , | , ^ , ~ , << , >> .
-Засвоїти чотири канонічні інженерні операції над апаратними регістрами мікропроцесора: встановлення, скидання, інвертування та тестування стану бітів за допомогою динамічно сформованих масок.
- Опанувати командний процес інтеграції коду через створення, рецензування та злиття Pull Request (PR) на платформі GitHub.
-Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

Індивідуальне завдання (Варіант 3) та опис масок

Завдання 1: Функція перевірки числа на простоту; локальна змінна зі специфікатором static підраховує загальну кількість виявлених простих чисел за сесію.

Завдання 2 (Робота з 32-бітним регістром):
Встановлення біта 4: reg | (1U << 4) (маска 0x00000010).
Скидання біта 8: reg & ~(1U << 8) (маска 0xFFFFFEFF).
Інвертування біта 14: reg ^ (1U << 14) (маска 0x00004000).
Перевірка стану біта 2: (reg >> 2) & 1U.

Завдання 3 (Пакування телеметрії у 16-бітне слово):
[Біти 0..3] — Пріоритет (0..15), маска 0x0F, зсув << 0.
[Біти 4..11] — Швидкість вентилятора (0..255), маска 0xFF, зсув << 4.
[Біти 12..13] — Код помилки (0..3), маска 0x03, зсув << 12.
[Біт 14] — Живлення (0..1), маска 0x01, зсув << 14.
Розрахунок контрольного прикладу: для значень 11 (0x0B), 185 (0xB9), 2 (0x02), 1 (0x01) отримуємо: (1 << 14) | (2 << 12) | (185 << 4) | 11 = 0x4000 | 0x2000 | 0x0B90 | 0x000B = 0x6B9B (27547 у десятковій системі).

Лістинги програм

task1.c

```c
#include <stdio.h>
#include <stdbool.h>

// Прототипи функцій
bool is_prime(int n, int *total_primes);
```

```c
int main(void) {
    int numbers[] = {2, 4, 7, 10, 13, 15, 17, 21, 29};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int prime_count = 0;
```

```c
    printf("=== Перевірка чисел на простоту (Варіант 3) ===\n");
    for (int i = 0; i < size; i++) {
        bool result = is_prime(numbers[i], &prime_count);
        printf("Число %2d -> %s (Всього знайдено простих: %d)\n",
               numbers[i],
               result ? "ПРОСТЕ" : "складене",
               prime_count);
    }
```

```
    return 0;
}
```

```
// Реалізація підпрограми (аргумент n передається за значенням)
bool is_prime(int n, int *total_primes) {
    // Статична змінна накопичує кількість знайдених простих чисел за
сесію
    static int session_primes = 0;
```

```
    if (n < 2) {
        if (total_primes != NULL) *total_primes = session_primes;
        return false;
    }
```

```
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            if (total_primes != NULL) *total_primes = session_primes;
            return false;
        }
    }
```

```
    session_primes++;
    if (total_primes != NULL) *total_primes = session_primes;
    return true;
}
```

task2.c

```
#include <stdio.h>
#include <stdint.h>

// Прототипи функцій
uint32_t set_bit(uint32_t reg, uint8_t bit);
uint32_t clear_bit(uint32_t reg, uint8_t bit);
uint32_t toggle_bit(uint32_t reg, uint8_t bit);
uint8_t  check_bit(uint32_t reg, uint8_t bit);
void     print_binary(uint32_t reg);
```

```
int main(void) {
    // Початковий стан регістра (встановимо біт 8 та біт 2 для
наочності)
    uint32_t reg = (1U << 8) | (1U << 2);
```

```
    printf("=== Робота з 32-бітним регістром (Варіант 3) ===\n");
    printf("Початковий стан:       ");
    print_binary(reg);
```

```
    // 1. Встановити біт 4
    reg = set_bit(reg, 4);
    printf("Після set_bit(4):      ");
```

```
    print_binary(reg);

    // 2. Скинути біт 8
    reg = clear_bit(reg, 8);
    printf("Після clear_bit(8):    ");
    print_binary(reg);

    // 3. Інвертувати біт 14
    reg = toggle_bit(reg, 14);
    printf("Після toggle_bit(14):  ");
    print_binary(reg);

    // 4. Перевірити стан біта 2
    uint8_t state = check_bit(reg, 2);
    printf("Стан біта 2 (check_bit): %u\n", state);

    return 0;
}

uint32_t set_bit(uint32_t reg, uint8_t bit) {
    return reg | (1U << bit);
}

uint32_t clear_bit(uint32_t reg, uint8_t bit) {
    return reg & ~(1U << bit);
}

uint32_t toggle_bit(uint32_t reg, uint8_t bit) {
    return reg ^ (1U << bit);
}

uint8_t check_bit(uint32_t reg, uint8_t bit) {
    return (reg >> bit) & 1U;
}

void print_binary(uint32_t reg) {
    for (int i = 31; i >= 0; i--) {
        printf("%u", (reg >> i) & 1U);
        if (i % 4 == 0 && i != 0) {
            printf(" ");
        }
    }
    printf(" (0x%08X)\n", reg);
}
```

task3.c

```
#include <stdio.h>
#include <stdint.h>

// Прототипи функцій
```

```c
uint16_t pack_telemetry(uint8_t priority, uint8_t fan_speed, uint8_t
error_code, uint8_t power);
void unpack_telemetry(uint16_t packet, uint8_t *priority, uint8_t
*fan_speed, uint8_t *error_code, uint8_t *power);
```

```c
int main(void) {
    uint8_t orig_priority   = 11;   // 0..15 (4 біти)
    uint8_t orig_fan_speed  = 185;  // 0..255 (8 бітів)
    uint8_t orig_error_code = 2;    // 0..3 (2 біти)
    uint8_t orig_power      = 1;    // 0..1 (1 біт)
```

```c
    printf("=== Пакування та розпакування телеметрії (Варіант 3)
===\n");
    printf("Початкові дані:\n");
    printf("  Пріоритет:            %u\n", orig_priority);
    printf("  Швидкість вентилятора:%u\n", orig_fan_speed);
    printf("  Код помилки:          %u\n", orig_error_code);
    printf("  Живлення:             %u\n\n", orig_power);
```

```c
    // Пакування
    uint16_t packed = pack_telemetry(orig_priority, orig_fan_speed,
orig_error_code, orig_power);
    printf("Запакований пакет: HEX: 0x%04X (DEC: %u)\n\n", packed,
packed);
```

```c
    // Розпакування
    uint8_t dec_priority, dec_fan_speed, dec_error_code, dec_power;
    unpack_telemetry(packed, &dec_priority, &dec_fan_speed,
&dec_error_code, &dec_power);
```

```c
    printf("Відновлені дані після розпакування:\n");
    printf("  Пріоритет:            %u\n", dec_priority);
    printf("  Швидкість вентилятора:%u\n", dec_fan_speed);
    printf("  Код помилки:          %u\n", dec_error_code);
    printf("  Живлення:             %u\n", dec_power);
```

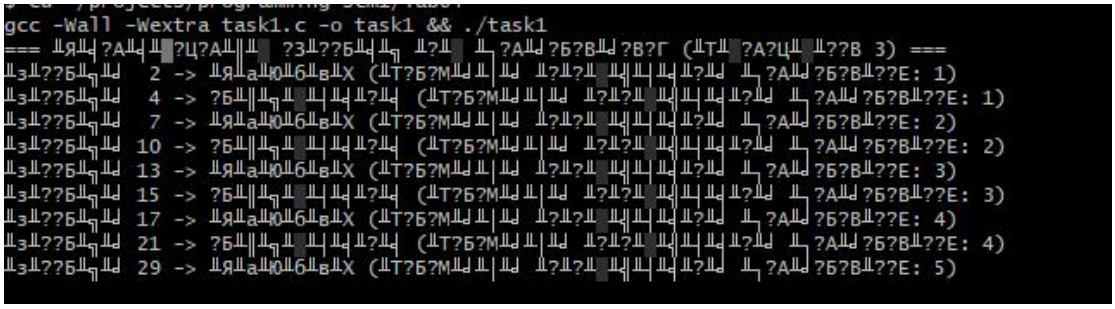
```c
    return 0;
}
```

```c
uint16_t pack_telemetry(uint8_t priority, uint8_t fan_speed, uint8_t
error_code, uint8_t power) {
    uint16_t packet = 0;
    packet |= (uint16_t)(priority   & 0x0F) << 0;   // Біти 0..3
    packet |= (uint16_t)(fan_speed  & 0xFF) << 4;   // Біти 4..11
    packet |= (uint16_t)(error_code & 0x03) << 12;  // Біти 12..13
    packet |= (uint16_t)(power      & 0x01) << 14;  // Біт 14
    return packet;
}
```

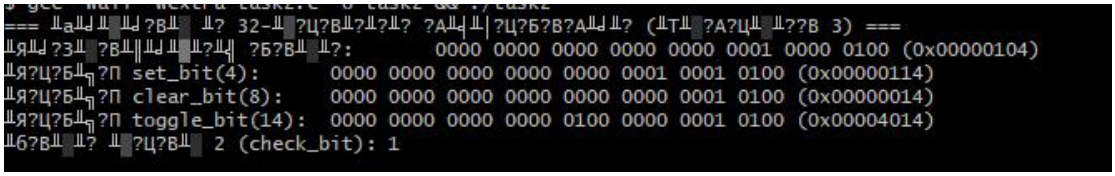
```c
void unpack_telemetry(uint16_t packet, uint8_t *priority, uint8_t
*fan_speed, uint8_t *error_code, uint8_t *power) {
    *priority   = (packet >> 0)  & 0x0F;
    *fan_speed  = (packet >> 4)  & 0xFF;
    *error_code = (packet >> 12) & 0x03;
    *power      = (packet >> 14) & 0x01;
}
```

Результати тестування

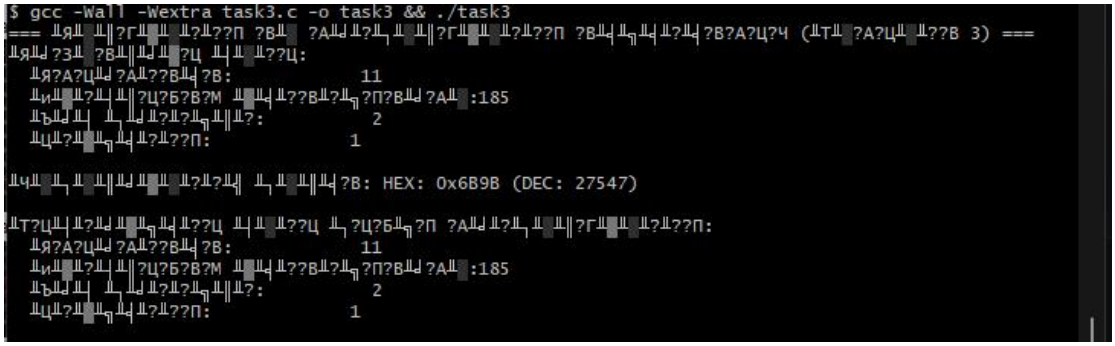
task1.c



task2.c



task3.c



Висновки

Під час виконання лабораторної роботи було реалізовано модульні програми мовою С із розділенням на прототипи та визначення функцій. Використання змінної зі специфікатором static забезпечило збереження лічильника простих чисел у статичній пам'яті між викликами без використання глобальних змінних. Застосування бітових зсувів та масок дозволило точково керувати окремими бітами 32-бітного регістра без впливу на сусідні розряди, а також виконати пакування та зворотне розпакування параметрів телеметрії у 16-бітне слово (0x6B9B). Роботу зафіксовано в окремій гілці lab04 та інтегровано в main через Pull Request на GitHub.